AFRILANG Stdlib

std/c/graphham

Français. Bibliothèque AFRILANG ‘std/c/graphham‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : hamiltonianExists, permuteCheck, longestPath, hamiltonianCycle, pathCoverSize, tspLowerBound, nearestNeighbor.
‘import ”std/c/graphham”‘ · ‘use graphham‘
- ‘hamiltonianExists(adj list number, n number) → bool‘ - ‘permuteCheck(adj list number, n number) → number‘ - ‘longestPath(adj list number, n number) → number‘ - ‘hamiltonianCycle(adj list number, n number) → bool‘ - ‘pathCoverSize(adj list number, n number) → number‘ - ‘tspLowerBound(adj list number, n number) → number‘ - ‘nearestNeighb **English.** AFRILANG library ‘std/c/graphham‘ (tier complex, category complex). Exports 7 function(s): hamiltonianExists, permuteCheck, longestPath, hamiltonianCycle, pathCoverSize, tspLowerBound, nearestNeighbor.
‘import ”std/c/graphham”‘ · ‘use graphham‘
- ‘hamiltonianExists(adj list number, n number) → bool‘ - ‘permuteCheck(adj list number, n number) → number‘ - ‘longestPath(adj list number, n number) → number‘ - ‘hamiltonianCycle(adj list number, n number) → bool‘ - ‘pathCoverSize(adj list number, n number) → number‘ - ‘tspLowerBound(adj list number, n number) → number‘ - ‘nearestNeighbor(adj list n

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/graphham‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : hamiltonianExists, permuteCheck, longestPath, hamiltonianCycle, pathCoverSize, tspLowerBound, nearestNeighbor.

‘import ”std/c/graphham”‘ · ‘use graphham‘

```
- `hamiltonianExists(adj list number, n number) → bool`
- `permuteCheck(adj list number, n number) → number`
- `longestPath(adj list number, n number) → number`
- `hamiltonianCycle(adj list number, n number) → bool`
- `pathCoverSize(adj list number, n number) → number`
- `tspLowerBound(adj list number, n number) → number`
- `nearestNeighbor(adj list number, n number, start number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphham"
use graphham
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 hamiltonianExists(adj list number, n number) → bool•
 permuteCheck(adj list number, n number) → number•
 longestPath(adj list number, n number) → number•
 hamiltonianCycle(adj list number, n number) → bool•
 pathCoverSize(adj list number, n number) → number•
 tspLowerBound(adj list number, n number) → number•
 nearestNeighbor(adj list number, n number, start number) → list
```

4. Exemples d’utilisation

```
import "std/c/graphham"
use graphham
```

```
create result = hamiltonianExists(/* args */)
say result
```

```
create result = permuteCheck(/* args */)
say result
```

```
create result = longestPath(/* args */)
say result
```

```
create result = hamiltonianCycle(/* args */)
say result
```

```
create result = pathCoverSize(/* args */)
say result
```

```
create result = tspLowerBound(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/graphham"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module graphham

```
function hamiltonianExists(adj list number, n number) returns bool
end
```

```
function permuteCheck(adj list number, n number) returns number
end
```

```
function longestPath(adj list number, n number) returns number
end
```

```
function hamiltonianCycle(adj list number, n number) returns bool
end
```

```
function pathCoverSize(adj list number, n number) returns number
end
```

```
function tspLowerBound(adj list number, n number) returns number
end
```

```
function nearestNeighbor(adj list number, n number, start number) returns list number
end
end
```

English

1. Overview

AFRILANG library ‘std/c/graphham‘ (tier complex, category complex). Exports 7 function(s): hamiltonianExists, permuteCheck, longestPath, hamiltonianCycle, pathCoverSize, tspLowerBound, nearestNeighbor.

‘import ”std/c/graphham”‘ · ‘use graphham‘

```
- `hamiltonianExists(adj list number, n number) → bool`
- `permuteCheck(adj list number, n number) → number`
- `longestPath(adj list number, n number) → number`
- `hamiltonianCycle(adj list number, n number) → bool`
- `pathCoverSize(adj list number, n number) → number`
- `tspLowerBound(adj list number, n number) → number`
- `nearestNeighbor(adj list number, n number, start number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphham"
use graphham
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 hamiltonianExists(adj list number, n number) → bool•
 permuteCheck(adj list number, n number) → number•
 longestPath(adj list number, n number) → number•
 hamiltonianCycle(adj list number, n number) → bool•
 pathCoverSize(adj list number, n number) → number•
 tspLowerBound(adj list number, n number) → number•
 nearestNeighbor(adj list number, n number, start number) → list
```

4. Usage examples

```
import "std/c/graphham"
use graphham
```

```
create result = hamiltonianExists(/* args */)
say result
```

```
create result = permuteCheck(/* args */)
say result
```

```
create result = longestPath(/* args */)
say result
```

```
create result = hamiltonianCycle(/* args */)
say result
```

```
create result = pathCoverSize(/* args */)
say result
```

```
create result = tspLowerBound(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/graphham"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module graphham

```
function hamiltonianExists(adj list number, n number) returns bool
end
```

```
function permuteCheck(adj list number, n number) returns number
end
```

```
function longestPath(adj list number, n number) returns number
end
```

```
function hamiltonianCycle(adj list number, n number) returns bool
end
```

```
function pathCoverSize(adj list number, n number) returns number
end
```

```
function tspLowerBound(adj list number, n number) returns number
end
```

```
function nearestNeighbor(adj list number, n number, start number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/graphham"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/graphham/

Source (extrait)

```
module graphham

function hamiltonianExists(adj list number, n number) returns bool
end

function permuteCheck(adj list number, n number) returns number
end

function longestPath(adj list number, n number) returns number
end

function hamiltonianCycle(adj list number, n number) returns bool
end

function pathCoverSize(adj list number, n number) returns number
end

function tspLowerBound(adj list number, n number) returns number
end

function nearestNeighbor(adj list number, n number, start number) returns list number
end
end
```